VASTRA: System Architecture & Design

Application Architecture Report | Component: High-Level System Design

This document defines the architectural blueprint for the VASTRA platform. It outlines the core system requirements, details the Azure-based data classification strategy, and explores the primary engineering trade-offs made during the implementation of the platform's microservices.

1. System Requirements

The system is designed to provide seamless AI-driven clothing discovery while ensuring high security, rapid response times, and robust cloud infrastructure.

Functional Requirements (What the system must do)	• **Visual Search:** Users must be able to upload outfit imagery to search for visually similar apparel across multiple external e-commerce platforms. • **Wishlist Management:** Authenticated users must be able to maintain, view, and modify a personalized repository of saved products. • **Verified Reviews:** The system must aggregate and display live customer reviews from external retailers (e.g., Amazon, Flipkart) using the Oxylabs API.
Non-Functional Requirements (How well the system must perform)	• **Security & IAM:** Secure authentication via Firebase with strict Role-Based Access Control (RBAC) enforced at the backend API routing layer. • **Latency:** AI styling analyses and recommendations must process and return a response within 3 seconds, utilizing asynchronous non-blocking I/O. • **Availability:** The application must maintain high availability, hosted on scalable Microsoft Azure App Service infrastructure.

2. Data Architecture (DP-900 Compliance)

The platform's storage topology is bifurcated into structured and unstructured data planes, strictly adhering to Azure DP-900 data classification principles to optimize cost, query performance, and scalability.

Structured Data (Azure SQL Database)	Core transactional entities—such as User profiles, Product metadata, and Wishlist mapping—require strict schemas, relational integrity, and ACID compliance. We provisioned an **Azure SQL Database** using a Third Normal Form (3NF) schema. This allows the enforcement of Foreign Key constraints between users and their saved products, and enables complex SQL `JOIN` operations and Stored Procedures for analytical querying.
Unstructured Data (Azure Blob Storage)	User-uploaded outfit images for AI analysis are raw binary files (JPEGs/PNGs) lacking a predefined relational data model. Storing these in a relational database would induce severe bloat and performance degradation. We utilized **Azure Blob Storage** (object storage). It provides infinite horizontal scalability, highly cost-effective storage for media, and immediate CDN-compatible URIs that our AI vision microservices can ingest directly.

3. Architectural Trade-off Analysis

In distributed systems engineering, the handling of third-party external data requires careful consideration between data freshness and system latency.

Decision: Live API Scraping vs. Persistent Database Caching

For the product review aggregation feature, we evaluated whether to scrape reviews periodically and cache them in the Azure SQL Database, or to fetch them live on-demand via the Oxylabs API.

- **Adopted Approach (Live Fetching):** We elected to retrieve reviews dynamically via decoupled, asynchronous React `useEffect` calls to the FastAPI backend, which proxies the request to the Oxylabs API.
- **Trade-off (The Cost):** This approach slightly increases the time-to-glass (latency) for the review section, as it is heavily dependent on external API response times and potential rate limits.
- **Strategic Rationale (The Benefit):** By accepting this latency penalty, we bypassed the need for massive database storage allocation for millions of external reviews. More importantly, it guarantees **100% data freshness** for the end-user, completely eliminating the complex engineering overhead of stale cache invalidation and background cron jobs.